** Content labeled as Internal, this info can only be shared within Vintecc! **

Secure Development Policy

ISO27001

Change History:

Date	Version	Author	Description
18/06/2025	1.0	Gilles Coene	Initial version

Contents

1. Introduction 3
2. References 3
3. Software development approaches 3
4. Security in the software development lifecycle 4
   4.1. Business requirements specification 4
   4.2. System design 5
   4.3. Development 5
   4.4. Testing 7
5. Security in outsourced development 7

1. Introduction

The purpose of this document is to set out rules regarding the development of software applications and components in a way which maximizes their inherent security.

2. References

The following controls are incorporated into this document:

- ISO 27001:2022 Annex A:
  - 8.25 – Secure development lifecycle
  - 8.26 – Application security requirements
  - 8.27 – Secure system architecture and engineering principles
  - 8.28 – Secure coding
  - 8.29 – Security testing in development
  - 8.30 – Outsourced development
  - 8.31 – Separation of development, test and production environments
  - 8.34 – Protection of information systems during audit and testing

3. Software development approaches

The process of software development fits in with the higher-level process of project management of new or enhanced information systems.
This process has the following major stages in a project:

- Proposal
- Planning
- Design and Development
- Transition
- Project Closure

The software development lifecycle sits mainly within the Design and Development stage and consists of the following sub-stages:

- Design and Development
  - Business requirements specification
  - System design
  - Development
  - Testing

At Vintecc, we adopt an agile methodology for software development, which contrasts with traditional waterfall models. We follow an iterative approach in our Secure Software Development Lifecycle (SSDLC). This includes Rapid Application Development (RAD), prototyping and agile frameworks such as Scrum. Furthermore, continuous integration/continuous delivery and DevOps practices are incorporated in our internal and external projects.

The Iterative approach typically involves significant stakeholder involvement throughout the development lifecycle and concentrates on producing frequent new versions of the software

that may be evaluated and tested before further functionality is added. The process loops round with each of the stages being carried out many times in small iterations (in the Agile method these are called “Sprints”). Approaches such as CI/CD (Continuous Integration/Continuous Delivery) and Continuous Deployment have evolved from the use of readily available cloud environments, and further reduce the time between code completion and its deployment.

While an iterative approach may be appropriate where exact requirements are less certain and frequent communication between developers and users (and within the development team) is possible, it could lead to security being overlooked in favour of rapid feature development. To ensure security remains a priority, Vintecc incorporates the following practices:

- Early and continuous security: Security requirements are defined early in the process and reviewed regularly throughout the development lifecycle to ensure they are maintained as the project evolves.
- Security Champion: A specific role assigned to someone in each of the development teams to promote secure coding practices and ensure security is integrated into every sprint.
- Ongoing developer training: Developers are continuously trained on secure coding standards and common vulnerabilities using internal and external training.
- DevSecOps: Security is embedded into the CI/CD pipeline of each project through automated testing and vulnerability scanning. It allows for quick remediation of issues before the software is “live”.

4. Security in the software development lifecycle

This section describes the way in which information security considerations must be incorporated into the various stages within the software development lifecycle.

4.1. Business requirements specification

The focus within the business requirements stage is on the functionality of the new software/system. This will be expressed in business rather than technical terms and should tie in with the business case that was produced prior to the initiation of the project.

The business is uniquely placed to give a clear understanding to the development team of the security requirements of the information that the new system will hold and process. In particular the business requirements must specify:

- The value of the information involved
- The sensitivity of the information – will personal data be held?
- The criticality of the new system and the information it holds – what is the business impact if it is not available?
- The legal, regulatory and contractual environment the system must operate within

A risk assessment must be carried out as part of the project to ensure that the implications of the above issues are fully understood by all parties.

4.2. System design

Based on the risk assessment and the classification of the information that is to be held in and processed by the new system, the design must provide for appropriate security features to be available.

Security requirements should be addressed across the following areas:

- User account and permission management
- Data input validation
- Data flow control
- Data output protection
- System interfaces
- Reporting
- Restart and recovery procedures
- Time stamps
- Logging (e.g. of transactions and access)
- Journaling of before and after images
- Batch and transaction counters
- Monitoring facilities
- How non-repudiation requirements will be met
- Ongoing patching arrangements
- Use of cryptography
- Need for digital certificates and signatures

Because of the iterative approach being used, the development teams must ensure these security controls are revisited and maintained with each iteration, ensuring that new changes do not compromise existing security measures.

4.3. Development

Before starting to write code, a secure development environment must be established for the project.

4.3.1. Secure development environment

To ensure the security and integrity of software development, the development environment, testing environment and production environment must be strictly separated.

Access control

Access to the development environment must be granted based on the least privilege principle according to the access control policy. Access rights should be regularly reviewed to ensure they remain appropriate.

Version control

Effective version control must be implemented from the start of the development including measures for source code check in/check out. This ensures traceability and accountability for all changes made to the code base.

Software versioning should be automated where possible using a version control tool that increments the version number whenever the item is checked in/out or changed.

Code monitoring

Where appropriate tools are available, amendments to the environment and the code stored within it should be monitored for unauthorized changes. This will help to pick up instances where development procedures are not being followed or fraudulent activity is suspected.

Backups

Regular backups of the development, testing and production environments must be performed to prevent data loss. These backups should be securely stored and periodically tested for integrity.

Secure repositories

Code and other program components should be stored in a secure repository to which access is restricted according to the projects access control policy.
An appropriate structure for the repository should be designed as part of the planning stage of the development.

Use of cloud repositories such as GitHub must be subject to the controls set out in the Supplier management policy.

4.3.2. Secure coding

To ensure the security of the software, appropriate secure coding guidelines must be adopted based on the development environment, programming languages, databases, tools and other technology components being used. These guidelines must be regularly reviewed and updated to defend against vulnerabilities and attacks, such as:

- Buffer overflow
- Time of Check/Time of Use
- Memory Reuse
- Malformed input
- SQL injection

For long-term projects, regular updates must be executed on newly discovered vulnerabilities and exploits associated with the technology stack in use. This helps maintain up-to-date protection against evolving threats.

Good practice in writing code will be followed at all times where appropriate, including:

- The use of structured programming techniques
- Clear documentation and commenting of code
- Consistent naming of items such as classes, methods and variables
- Avoiding hard coding of credentials
- Correct handling of errors

4.4. Testing

Throughout the software development lifecycle, various forms of testing are carried out, such as unit testing, system testing, integration testing, performance testing, user acceptance testing and operational acceptance testing. While security controls are assessed to some extent during these testing phases, it's important to conduct dedicated security testing to validate the security requirements established during the business requirements and design phase.

Security testing must be conducted with the same rigor and formality as other testing activities. Different security strategies are included in the SSDLC:

- Secret scanning
- Vulnerability scanning
- SBOM generation
- Pre-project security screening
- Software testing

Adequate controls must be put in place to protect test data. Where appropriate (and with prior approval on each occasion), a live to test copy may be made in order to provide representative test data. However, if this contains sensitive data such as personal data this must be removed or obscured before use.

In a CI/CD (Continuous Integration/Continuous Delivery) or Continuous Deployment scenario, in which the testing and deployment of code to the cloud may be automated, testing frameworks such as the OWASP (Open Web Application Security Project) Application Security Verification Standard (ASVS) must be used as a basis for testing application technical security controls. Appropriate SAST, DAST and container scanning tools may also be used where relevant.

5. Security in outsourced development

Where software development is wholly or partially outsourced to a third party, due care must be taken to ensure that the policies of Vintecc are still followed where possible. Vintecc will remain legally responsible for the use of the software created and the information contained within it even though it didn’t write the software. Therefore, the same level of rigour must be applied to outsourced software development as that created in-house.

The supplier management policy must be followed in case of outsourced development.